

Ejercicio Demostrativo

Nombre del Auxiliar

Tiempo estimado: XX hrs/min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Nombre del Curso - Sección]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Precisión	La precisión se evidencia cuando el estudiante escribe XML bien formado con etiquetas correctamente cerradas, valida la estructura del documento antes de procesarlo y verifica que los datos extraídos coincidan exactamente con los valores esperados.

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Procesar e intercambiar datos estructurados en formato XML utilizando las librerías del ecosistema .NET en C#, aplicando técnicas de lectura, escritura y consulta mediante XmlDocument, XDocument y XPathNavigator para resolver problemas reales de manejo de datos.
Competencia Específica	Leer y escribir archivos XML en C# utilizando XmlDocument y XDocument, navegar y consultar nodos mediante XPathNavigator y expresiones XPath, y crear o modificar documentos XML desde código de forma programática.

2. Palabras Clave

XML (eXtensible Markup Language): Lenguaje de marcado que permite estructurar datos de forma jerárquica usando etiquetas personalizadas. XmlDocument: Clase de .NET del espacio de nombres System.Xml que permite cargar, navegar y manipular documentos XML mediante el modelo DOM. XDocument: Clase de LINQ to XML (System.Xml.Linq) que ofrece una API más moderna y funcional para trabajar con XML en C#. XPath: Lenguaje de consulta para seleccionar nodos y valores dentro de un documento XML usando expresiones de ruta. XPathNavigator: Clase de .NET que permite navegar por un documento XML usando expresiones XPath para localizar nodos específicos. Nodo XML: Unidad básica de un documento XML; puede ser un elemento, atributo, texto, comentario u otro tipo de componente.

3. Resumen

Este ejercicio demostrativo presenta de forma guiada el procesamiento de datos XML en C#, cubriendo desde la estructura básica de un documento XML hasta su lectura con XmlDocument y XDocument, la navegación mediante XPathNavigator, y la lectura y escritura programática de archivos XML.

La clase se orienta a que el estudiante cargue un archivo XML real, extraiga datos usando distintas librerías (.NET), consulte nodos con expresiones XPath y genere un nuevo archivo XML desde código, verificando en cada paso que los datos procesados sean correctos.

4. Contenido

Descripción del ejercicio

Este ejercicio consiste en procesar un archivo XML de actividades de laboratorio usando las librerías XmlDocument, XDocument y XPathNavigator de C#. Se leerá el archivo, se extraerán datos específicos y se escribirá un nuevo XML con resultados modificados. Al finalizar, el estudiante tendrá un programa en C# que demuestre lectura, consulta y escritura de datos XML usando las tres librerías.

Objetivo de la demostración

Demostrar el procedimiento para leer, consultar y escribir documentos XML en C#, utilizando XmlDocument, XDocument y XPathNavigator, con el fin de que el estudiante comprenda cómo integrar datos estructurados XML en aplicaciones reales.

Recursos necesarios

Recursos utilizados en la demostración:

- Software: Visual Studio Community 2022 o VS Code con extensión C#. .NET SDK 8.0 o superior.
- Herramientas técnicas: Librerías System.Xml y System.Xml.Linq de .NET, editor de XML integrado en Visual Studio, visor de XML del navegador para validación visual.
- Materiales de apoyo: Archivo actividades.xml de ejemplo con estructura definida, referencia de métodos de XmlDocument, XDocument y XPathNavigator.
- Equipo: Computadora con .NET SDK instalado. No se requiere conexión a internet si los paquetes NuGet ya están disponibles localmente.

- Configuración: Proyecto de tipo consola (.NET). Agregar using System.Xml; y using System.Xml.Linq; al inicio del archivo de código.

Desarrollo paso a paso

Nota: Describe el procedimiento en orden lógico y operativo, recuerda que esto será utilizado por otro tutor ya que tu práctica dura dos semestres y en el siguiente estará otro tutor posiblemente.

Cada paso debe redactarse con claridad, evitando explicaciones innecesarias, pero incluyendo lo suficiente para que se entienda qué se hace, por qué se hace y qué resultado parcial se espera.

Paso 1

Introducción a XML y lectura con XmlDocument.

Crear el archivo actividades.xml con estructura jerárquica (elementos, atributos y texto). Cargar el archivo con XmlDocument usando xmlDoc.Load("actividades.xml") y recorrer los nodos con GetElementsByTagName(). El estudiante debe observar cómo se accede a cada nodo y su valor de texto.

Paso 2

Procesamiento de datos con XDocument (LINQ to XML).

Cargar el mismo archivo con XDocument.Load() y usar consultas LINQ (doc.Root.Elements()) para extraer datos. Comparar la sintaxis con XmlDocument: XDocument ofrece una API más legible y compatible con LINQ. Verificar que los datos extraídos sean idénticos a los obtenidos con XmlDocument.

Paso 3

Navegación con XPathNavigator y consultas XPath.

Crear un XPathNavigator desde el XmlDocument con CreateNavigator(). Usar expresiones XPath como "//actividad[@id='1']/nombre" para localizar nodos específicos. Demostrar Select(), SelectSingleNode() y Evaluate(). El estudiante debe verificar que la expresión XPath retorna exactamente el nodo esperado.

Paso 4

Lectura y escritura de XML: generar un nuevo archivo XML desde código.

Usar XmlWriter o XmlDocument para crear un archivo resultado.xml con los datos procesados. Guardar con xmlDoc.Save() o xdoc.Save(). Abrir el archivo generado y verificar que la estructura XML sea válida y los datos correspondan a los procesados. El ejercicio está completo cuando resultado.xml abre sin errores en el navegador o en Visual Studio.

Observaciones importantes

). Manejar excepciones con try-catch al cargar archivos XML (XmlException). Las expresiones XPath son sensibles a mayúsculas. Preferir XmlDocument sobre XmlDocument para proyectos nuevos por su integración con LINQ.

Tabla 1

Comparativa de librerías para procesamiento XML en C#:

Elemento	Descripción
XmlDocument	DOM clásico, compatible con todas las versiones de .NET. Usa GetElementsByTagName() y SelectNodes() con XPath
XDocument	API moderna con LINQ to XML. Más legible y funcional. Usa XDocument.Load(), Root.Elements() y consultas LINQ
XPathNavigator	Navegador de cursor sobre el DOM. Usa expresiones XPath con Select(), SelectSingleNode() y Evaluate() para consultas avanzadas

Resultado esperado

Al ejecutar correctamente el ejercicio, el programa debe mostrar en consola los datos extraídos del XML usando las tres librerías y generar un archivo resultado.xml válido.

El archivo resultado.xml debe poder abrirse en un navegador sin errores de parseo, y los valores mostrados en consola deben coincidir con los datos del archivo original actividades.xml.

Aplicación práctica

El procesamiento de XML es fundamental en la integración de sistemas: servicios web SOAP, archivos de configuración (.config), intercambio de datos entre aplicaciones y lectura de feeds RSS/Atom. En el ámbito profesional, estas habilidades se aplican en el desarrollo de APIs, migración de datos y configuración de aplicaciones .NET empresariales.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

- Dificultades encontradas: Los estudiantes frecuentemente olvidan el espacio de nombres (namespace) en los documentos XML, lo que hace que las expresiones XPath no encuentren los nodos. También confunden los métodos de XmlDocument con los de XDocument.
- Recomendaciones: Preparar un archivo XML de ejemplo con datos familiares para los estudiantes (ej. lista de cursos o estudiantes). Mostrar el XML en el navegador antes de procesarlo para que visualicen la estructura jerárquica.
- Mayor dificultad: Construir expresiones XPath correctas para nodos con atributos, y entender la diferencia práctica entre XmlDocument y XDocument al momento de modificar el documento.

6. Referencias

1. Microsoft. (2024). System.Xml Namespace - Documentación de .NET. <https://learn.microsoft.com/es-es/dotnet/api/system.xml>
2. Price, M. J. (2023). C# 12 and .NET 8 - Modern Cross-Platform Development. Cap. XML & JSON. Packt Publishing.